EXP NO: 3	Comparison Of Sgd With Momentum Vs Adam Optimizer

AIM:

To implement a training algorithm using Stochastic Gradient Descent (SGD) with momentum, train the same network with the Adam optimizer on the CIFAR-10 dataset, and compare the convergence rate and performance of the two optimizers.

ALGORITHM:

1. Load CIFAR-10, normalize the images with ToTensor and Normalize(0.5, 0.5), and use the first 10,000 training images with batch size 128.
2. Define a Multi-Layer Perceptron: Flatten, Linear(3072, 256), ReLU, Linear(256, 128), ReLU, Linear(128, 10).
3. Write a training function that, for each epoch, performs the forward pass, computes the cross-entropy loss, backpropagates, updates the weights and records the average loss and accuracy.
4. Train one MLP with SGD (learning rate 0.01, momentum 0.9) for 5 epochs.
5. Train a fresh MLP with Adam (learning rate 0.001) for 5 epochs.
6. Plot the loss and accuracy curves of both optimizers and compare their final accuracies.

CODE:

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt

# Device
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)
print("Using device:", device)

# CIFAR-10 preprocessing transform =
transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(
        (0.5, 0.5, 0.5),
        (0.5, 0.5, 0.5)
    )
])

# Download CIFAR-10
trainset = torchvision.datasets.CIFAR10(
    root="./data",
    train=True,
    download=True,
    transform=transform
)

# Use smaller dataset for faster lab execution trainset
= torch.utils.data.Subset(
    trainset,
    range(10000)
)
```

```
trainloader = torch.utils.data.DataLoader(
        trainset,
        batch_size=128,
        shuffle=True
)

# MLP model
class MLP(nn.Module):
        def __init__(self):
                super().__init__()
                self.flatten = nn.Flatten()
                self.network = nn.Sequential(
                        nn.Linear(3 * 32 * 32, 256),
                        nn.ReLU(),
                        nn.Linear(256, 128),
                        nn.ReLU(),
                        nn.Linear(128, 10)
                )
                def forward(self, x):
                        x = self.flatten(x)
                        return self.network(x)

# Training function
def train_model(model, optimizer, epochs=5):
        model = model.to(device)
        criterion = nn.CrossEntropyLoss()
        losses = []
        accuracies = []
for epoch in range(epochs):
        model.train()
        total_loss = 0
        correct = 0
        total = 0
        for images, labels in trainloader:
        images = images.to(device)
        labels = labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(
                        outputs,
                        labels
                )
                loss.backward()
                optimizer.step() total_loss +=
                loss.item()   _,   predicted   =
                torch.max(
                        outputs,
                        1
                )
                total += labels.size(0)
                correct += (
                        predicted == labels
                ).sum().item()
average_loss = (
        total_loss / len(trainloader)
)
accuracy = (
        100 * correct / total
)     losses.append(average_loss)
accuracies.append(accuracy)
print(

        "Epoch {}/{} - Loss: {:.4f} - Accuracy: {:.2f}%".format(
```

```
            epoch + 1,
            epochs,
            average_loss,
            accuracy
        )
)
return losses, accuracies

# SGD with Momentum
print("\n=====     SGD     WITH     MOMENTUM
=====") model_sgd = MLP()
optimizer_sgd = optim.SGD(
        model_sgd.parameters(),
        lr=0.01,
        momentum=0.9
)
loss_sgd, acc_sgd = train_model(
        model_sgd,
        optimizer_sgd
)

# Adam
print("\n===== ADAM =====")
model_adam          =          MLP()
optimizer_adam = optim.Adam(
        model_adam.parameters(),
        lr=0.001
)
loss_adam, acc_adam = train_model(
        model_adam,
        optimizer_adam
)

#          Loss          comparison
plt.figure(figsize=(8,             5))
plt.plot(
        loss_sgd,
        marker="o",
        label="SGD + Momentum"
) plt.plot(

        loss_adam,
        marker="o",
        label="Adam"
)
plt.xlabel("Epoch")
plt.ylabel("Loss")      plt.title("Loss
Comparison") plt.legend()
plt.grid()
plt.show()

#         Accuracy         comparison
plt.figure(figsize=(8, 5)) plt.plot(



        acc_sgd,
        marker="o",
        label="SGD + Momentum"
) plt.plot(

        acc_adam,
        marker="o",
        label="Adam"
```

```
)
plt.xlabel("Epoch")   plt.ylabel("Accuracy   (%)")
plt.title("Accuracy Comparison") plt.legend()
plt.grid()
plt.show()

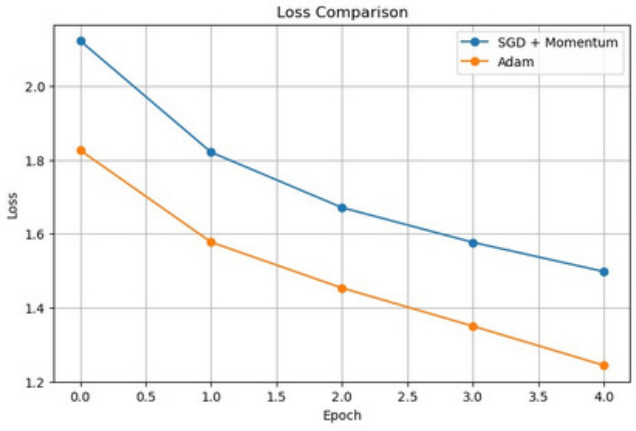
# Final comparison
print("\n=====          FINAL          COMPARISON
=====") print(

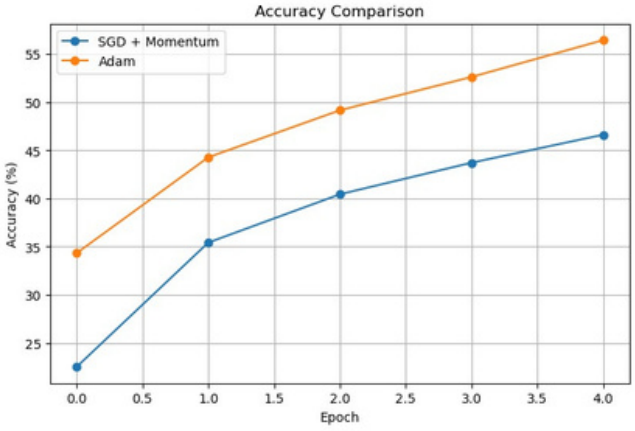

      "SGD + Momentum Final Accuracy:",
      round(acc_sgd[-1], 2),
      "%"
) print(

      "Adam    Final    Accuracy:",
      round(acc_adam[-1],       2),
      "%"
)
```

OUTPUT:

```
===== SGD WITH MOMENTUM =====
Epoch 1/5 - Loss: 2.1227 - Accuracy: 22.51% Epoch
2/5 - Loss: 1.8208 - Accuracy: 35.43% Epoch 3/5 -
Loss: 1.6710 - Accuracy: 40.45% Epoch 4/5 - Loss:
1.5768 - Accuracy: 43.73% Epoch 5/5 - Loss: 1.4978
- Accuracy: 46.64% ===== ADAM =====
Epoch 1/5 - Loss: 1.8262 - Accuracy: 34.33% Epoch
2/5 - Loss: 1.5773 - Accuracy: 44.29% Epoch 3/5 -
Loss: 1.4533 - Accuracy: 49.18% Epoch 4/5 - Loss:
1.3500 - Accuracy: 52.64% Epoch 5/5 - Loss: 1.2437
- Accuracy: 56.47% ===== FINAL COMPARISON
=====
SGD + Momentum Final Accuracy: 46.64 %
Adam Final Accuracy: 56.47 %
```





RESULT:

SGD with momentum and Adam were compared on CIFAR-10 for 5 epochs. Adam converged faster and reached a final training loss of 1.2437 and training accuracy of 56.47%, whereas SGD with momentum reached a loss of 1.4978 and an accuracy of 46.64%.